Jean Victor Borges de Abreu

Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Petrópolis - Rio de Janeiro
2026

Jean Victor Borges de Abreu

Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Trabalho de Conclusão de Curso apresentado à...

FACULDADE DE EDUCAÇÃO TECNOLÓGICA DO ESTADO DO RIO DE JANEIRO
FAETERJ/PETRÓPOLIS

Orientador: Nome do Orientador

Petrópolis - Rio de Janeiro
2026

1 Resumo

2 Abstract

3 Lista de Figuras

4 Lista de Abreviaturas

FAETERJ: Faculdade de Educação Tecnológica do Estado do Rio de Janeiro

Sumário

1 RESUMO 2

2 ABSTRACT 3

3 LISTA DE FIGURAS 4

4 LISTA DE ABREVIATURAS 5

5 INTRODUÇÃO 7
5.1 Contextualização do Tema 7
5.2 Justificativa/Problema de Pesquisa 7
5.3 Motivação/Hipotese e Ameaca Tecnológica 7
5.4 Objetivos 8
5.5 Metodologia 8

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO . 9
6.1 Aleatoriedade Verdadeira vs. Pseudoaleatoriedade 9
6.2 Entropia e Fontes de Incerteza 9
6.3 A Semente (Seed) como Elemento Crítico 9
6.4 Avaliação Estatística da Aleatoriedade 9

7 GERADORES CRIPTOGRAFICAMENTE SEGUROS 10
7.1 RNGs vs. CSPRNGs 10
7.2 O Ataque do Próximo Bit 10
7.2.1 Implementação Prática (Demonstração) 10
7.3 Implementação de CSPRNGs em Java 10
7.4 Falhas Históricas de Aleatoriedade 10

8 COMPUTAÇÃO QUÂNTICA E A QUEBRA DO PARADIGMA ATUAL 11
8.1 Conceitos Fundamentais de Computação Quântica 11
8.2 Algoritmo de Shor 11

8.3 Algoritmo de Grover 11
8.4 Harvest Now, Decrypt Later 11

9 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC) 12
9.1 Definição e Objetivos da PQC 12
9.2 Paradigmas Matemáticos da PQC 12
9.3 Criptografia Baseada em Reticulados 12
9.4 Learning With Errors (LWE) 12
9.4.1 Implementação Didática em Java (Demonstração) 12
9.5 Padronização Pós-Quântica 12

10 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA 13
10.1 Suporte à Criptografia Pós-Quântica no Java 13
10.2 Crypto-Agility 13
10.2.1 Implementação Prática: Demo de Crypto-Agility 13
10.3 Comparativo de Modelos 13
10.3.1 Implementação Prática: Benchmark Clássico vs. Kyber 13

11 DISCUSSÃO E ANÁLISE CRÍTICA 14
11.1 Impacto em Desempenho 14
11.2 Barreiras de Adoção 14
11.3 Implicações Sociais e Éticas 14

12 DISCUSSÃO E ANÁLISE CRÍTICA 15
12.1 Síntese dos Resultados 15
12.2 Resposta ao Problema de Pesquisa 15
12.3 Trabalhos Futuros 15

13 COISAS POSSIVELMENTE UTEIS EM ALGUM MOMENTO . . 16

14 CONCLUSÃO 18

5 Introdução

A infraestrutura de segurança digital de grandes corporações, em sua maioria sustentada por ecossistemas como o Java, vivencia um momento de transição crítica. Visto que computadores operam sob lógica determinística e não podem gerar o 'acaso puro' sem fontes externas de entropia, a criptografia contemporânea torna-se dependente da robustez matemática dos Geradores de Números Pseudo Aleatórios (PRNGs).

Durante décadas, bibliotecas de segurança confiaram em implementações consolidadas, como a classe java.security.SecureRandom, para garantir a integridade de chaves e certificados. Contudo, o setor presencia o ocaso da criptografia clássica: a padronização recente liderada pelo NIST indica que as fontes de entropia e os algoritmos de pseudo aleatoriedade tradicionais não são mais suficientes contra ataques baseados em vetores quânticos. Nesse cenário, se o gerador de números responsável pela criação de chaves for previsível para um sistema quântico, a estrutura de segurança da informação como um todo torna-se obsoleta.

O presente artigo explora esse panorama de mudança, avaliando como a transição para a Criptografia Pós-Quântica (PQC) altera as práticas de desenvolvimento e investigando novos padrões de redes de pontos (lattices), bem como a adaptação da plataforma Java para assegurar a soberania dos dados em um futuro pós-quântico.

5.1 Contextualização do Tema

Com o avanço da tecnologia e o uso crescente de ferramentas e hardwares quanticos. A utilização de meios pesudoaleatorios tem sidos colocados em risco. E sua utilização em alguns anos podem se tornar obsoletos e inseguros para utilização. Como já é o caso de grandes empresas e organizações governamentais. Das quais já fazem uso de outros meios para criptografar suas informações.

5.2 Justificativa/Problema de Pesquisa

Muitos Textos

5.3 Motivação/Hipotese e Ameaca Tecnológica

Muitos textos

5.4 Objetivos

Muitos textos

5.5 Metodologia

Muitos textos

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO

6.1 Aleatoriedade Verdadeira vs. Pseudoaleatoriedade

Muitos textos

6.2 Entropia e Fontes de Incerteza

Muitos textos

6.3 A Semente (Seed) como Elemento Crítico

Muitos textos

6.4 Avaliação Estatística da Aleatoriedade

Muitos textos

7 Geradores Criptograficamente Seguros

7.1 RNGs vs. CSPRNGs

Muitos textos

7.2 O Ataque do Próximo Bit

Muitos textos

7.2.1 Implementação Prática (Demonstração)

Muitos textos

7.3 Implementação de CSPRNGs em Java

Muitos textos

7.4 Falhas Históricas de Aleatoriedade

Muitos textos

8 COMPUTAÇÃO QUÂNTICA E A QUEBRA DO PARADIGMA ATUAL

8.1 Conceitos Fundamentais de Computação Quântica

Muitos textos

8.2 Algoritmo de Shor

Muitos textos

8.3 Algoritmo de Grover

Muitos textos

8.4 Harvest Now, Decrypt Later

Muitos textos

9 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC)

9.1 Definição e Objetivos da PQC

Muitos textos

9.2 Paradigmas Matemáticos da PQC

Muitos textos

9.3 Criptografia Baseada em Reticulados

Muitos textos

9.4 Learning With Errors (LWE)

Muitos textos

9.4.1 Implementação Didática em Java (Demonstração)

Muitos textos

9.5 Padronização Pós-Quântica

Muitos textos

10 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA

10.1 Suporte à Criptografia Pós-Quântica no Java

Muitos textos

10.2 Crypto-Agility

Muitos textos

10.2.1 Implementação Prática: Demo de Crypto-Agility

Muitos textos

10.3 Comparativo de Modelos

Muitos textos

10.3.1 Implementação Prática: Benchmark Clássico vs. Kyber

Muitos textos

11 DISCUSSÃO E ANÁLISE CRÍTICA

11.1 Impacto em Desempenho

Muitos textos

11.2 Barreiras de Adoção

Muitos textos

11.3 Implicações Sociais e Éticas

Muitos textos

12 DISCUSSÃO E ANÁLISE CRÍTICA

12.1 Síntese dos Resultados

Muitos textos

12.2 Resposta ao Problema de Pesquisa

Muitos textos

12.3 Trabalhos Futuros

Muitos textos

13 Coisas possivelmente Uteis em algum momento

PRNG é uma sigla para “Pseudo Random Number Generator” ou “Gerador de Números Pseudo Aleatórios”. Seu significado é que a sua geração de números não segue uma estrutura aleatória. Ele utiliza um ponto inicial denominado “seed” para dar início a sua geração de números. A seed é um valor base para ser utilizado no PRNG. Utilizando a mesma seed, o resultado final do PRNG será o mesmo.

O PRNG funciona a partir de uma função que recebe a seed por parâmetro. E através de um cálculo, ele retorna o resultado dessa fórmula. Uma das fórmulas mais comuns para esse tipo de gerador de pseudo números aleatórios é no modelo LCG “Gerador Congruencial Linear”

$$\boldsymbol{x_{i+1} = (A \cdot x_i + C) \pmod{M}} \tag{13.1}$$

Onde o valor inicial “x0” é a seed, A e C são parâmetros inteiros que podem ter um valor já pré-definido ou passados diretamente, M é o espaço dos valores aleatórios possíveis e “mod” é o restante da divisão inteira.

Figura 1 – Modelo simples de um PRNG com LCG

```java
public class Main {
    public static void main(String[] args) {
        long seed = 12345;
        long a = 1664525;
        long c = 1013904223;
        long mod = (long) Math.pow(2, 32);

        LCG lcg = new LCG(seed, a, c, mod);

        for (int i = 0; i < 10; i++) {
            System.out.println(lcg.next());
        }

    }

    public static class LCG {  2 usages

        private long a;  2 usages
        private long c;  2 usages
        private long mod;  2 usages
        private long xi;  4 usages

        public LCG(long seed, long a, long c, long mod) {  1 usage
            this.xi = seed;
            this.a = a;
            this.c = c;
            this.mod = mod;
        }

        public long next() {  1 usage
            xi = (a * xi + c) % mod;
            return xi;
        }
    }
}
```

Fonte: Elaborado pelo autor (2026)

Mesmo após inúmeras tentativas. Devido a seed permanecer inalterada. O resultado permanece o mesmo em todos os resultados obtidos.

Por mais que tenha se passado um valor fixo na seed no exemplo prático, a seed é definida pelo sistema operacional, esse número é denominado entropia. Que se tem por referência a quantidade de imprecisão de um sistema. Quanto maior o número da entropia, maior é a incerteza. Esse valor de entropia é gerado a partir de certos eventos que ocorrem no momento de sua captura, como por exemplo o movimento do mouse, os inputs no teclado etc. Essas capturas são convertidas em um valor numérico para ser usado na seed.

Esses inputs que o sistema captura podem ser previsíveis, tornando a entropia causada menor. Sendo possível pré-determinar o valor gerado ao final para seed. Apesar das aplicações para o dia a dia ou em pequenos projetos serem o suficiente. Esse método não é seguro para aplicações de alto risco.

Figura 2 – Resultado do script LCG

```
/app/extra/jbr/bin/java -javaagent:/app/extra/lib/idea_rt.jar=34643 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encodin
87628868
71072467
2332836374
2726892157
3908547000
483019191
2129828778
2355140353
2560230508
3364893915

Process finished with exit code 0
```

Fonte: Elaborado pelo autor (2026)

14 Conclusão

Considerações finais.